

Interview Platform - RBAC Setup Guide

Overview

This document contains all the required setup data for:

- Roles
- Permissions
- Role Permissions Mapping
- Default User Role Assignment

If database data is deleted/reset, use these APIs and payloads to recreate the required master data.

Architecture Overview

User

↓

UserRole

↓

Role

↓

RolePermission

↓

Permission

Roles

Role	Description
------	-------------

---	---
-----	-----

ADMIN	Full system access
-------	--------------------

INTERVIEWER	Conducts interviews
-------------	---------------------

CANDIDATE	Attends interviews
-----------	--------------------

RECRUITER	Schedules/manages interviews
-----------	------------------------------

Permissions

Permission	Description
------------	-------------

---	---
-----	-----

CREATE_INTERVIEW	Create interview sessions
UPDATE_INTERVIEW	Update interview details
DELETE_INTERVIEW	Delete interviews
JOIN_INTERVIEW	Join interview session
SUBMIT_FEEDBACK	Submit interview feedback
MANAGE_USERS	Manage users and roles
VIEW_REPORTS	View analytics/reports

Step 1 — Create Roles

Endpoint:

POST /api/v1/roles

ADMIN

```
{  
  "name": "ADMIN",  
  "description": "Full system access"  
}
```

INTERVIEWER

```
{  
  "name": "INTERVIEWER",  
  "description": "Can conduct interviews"  
}
```

CANDIDATE

```
{  
  "name": "CANDIDATE",  
  "description": "Can attend interviews"  
}
```

RECRUITER

```
{  
  "name": "RECRUITER",  
  "description": "Can manage interview scheduling"  
}
```

Step 2 — Create Permissions

Endpoint:

POST /api/v1/permissions

CREATE_INTERVIEW

```
{  
  "name": "CREATE_INTERVIEW",  
  "description": "Can create interviews"  
}
```

UPDATE_INTERVIEW

```
{  
  "name": "UPDATE_INTERVIEW",  
  "description": "Can update interviews"  
}
```

DELETE_INTERVIEW

```
{  
  "name": "DELETE_INTERVIEW",  
  "description": "Can delete interviews"  
}
```

JOIN_INTERVIEW

```
{  
  "name": "JOIN_INTERVIEW",  
  "description": "Can join interview sessions"  
}
```

SUBMIT_FEEDBACK

```
{  
  "name": "SUBMIT_FEEDBACK",  
  "description": "Can submit interview feedback"  
}
```

MANAGE_USERS

```
{  
  "name": "MANAGE_USERS",  
  "description": "Can manage users"  
}
```

VIEW_REPORTS

```
{  
  "name": "VIEW_REPORTS",  
  "description": "Can view reports and analytics"  
}
```

Step 3 — Assign Permissions To Roles

Endpoint:

POST /api/v1/roles/{roleId}/permissions

Request Body:

```
{  
  "permissionId": "permission-uuid"  
}
```

Recommended Role Permission Mapping

ADMIN:

- CREATE_INTERVIEW
- UPDATE_INTERVIEW
- DELETE_INTERVIEW
- JOIN_INTERVIEW
- SUBMIT_FEEDBACK
- MANAGE_USERS
- VIEW_REPORTS

INTERVIEWER:

- CREATE_INTERVIEW
- UPDATE_INTERVIEW
- JOIN_INTERVIEW
- SUBMIT_FEEDBACK
- VIEW_REPORTS

CANDIDATE:

- JOIN_INTERVIEW

RECRUITER:

- CREATE_INTERVIEW
- UPDATE_INTERVIEW
- VIEW_REPORTS

Step 4 — Create User

Endpoint:

POST /api/v1/users

Request Body:

```
{
  "firstName": "Arjun",
  "lastName": "Reddy",
  "email": "arjun.reddy@example.com",
  "password": "Password@123",
  "phoneNumber": "9876543210"
}
```

Internal Signup Flow

1. User created
2. Empty profile created
3. Default role assigned -> CANDIDATE
4. user_roles populated

Important Tables

- users
- user_profiles
- roles
- permissions

- user_roles
- role_permissions

Notes

- Never directly assign permissions to users.
- Assign permissions only to roles.
- Users inherit permissions through assigned roles.
- Prefer explicit join entities (`UserRole`, `RolePermission`) over direct `ManyToMany`.